



Universidade do Minho

Licenciatura em Engenharia Informática

Relatório da Fase 1 – Laboratórios de Informática III

Ano Letivo de 2025/2026

Grupo 024

Sara Marques, a111182;

Marta Araújo, a110317;

Teresa Teixeira, a111415.

Índice

1. Introdução
2. Trabalho desenvolvido em geral
3. Arquitetura do Sistema
4. Discussão e Análise Crítica
5. Conclusão

1. Introdução

O presente relatório descreve o trabalho desenvolvido no âmbito da Fase 1 do projeto prático da unidade curricular de Laboratórios de Informática III, desenvolvido pelo grupo 24 constituído por: Sara Marques, Marta Araújo e Teresa Teixeira.

O objetivo central deste projeto consistiu na implementação de um sistema de gestão de informação relativo a aeroportos, voos, aeronaves, passageiros e reservas.

O programa foi desenvolvido na linguagem C e é capaz de carregar dados a partir de ficheiros no formato CSV, submetê-los a um processo de validação sintática e lógica, e armazená-los em estruturas de dados em memória, preparando assim para a subsequente execução de interrogações, as queries, sobre essa informação.

2. Trabalho desenvolvido em geral

Nesta primeira fase, foi dada atenção à implementação de aspetos fundamentais como a modularidade e o encapsulamento. O sistema foi estruturado em vários módulos, separando claramente as responsabilidades de leitura de dados, validação, gestão de entidades e execução de queries. Um dos aspetos mais relevantes do trabalho desenvolvido reside no mecanismo de validação implementado, que assegura a integridade dos dados carregados, rejeitando registos inválidos e registrando os respectivos erros em ficheiros.

Adicionalmente, foi estabelecido um módulo de testes automáticos, que permite determinar com rigor o funcionamento correto de todo o sistema.

3. Arquitetura do Sistema

Seguindo os princípios da modularidade, o programa está organizado em componentes bem definidos que interagem entre si.

O módulo principal, 'programa-principal' inicia o processo de carregamento dos dados.

A leitura e o parsing dos ficheiros CSV são da responsabilidade do módulo 'parser', que percorre cada linha dos ficheiros de entrada e extrai os dados.

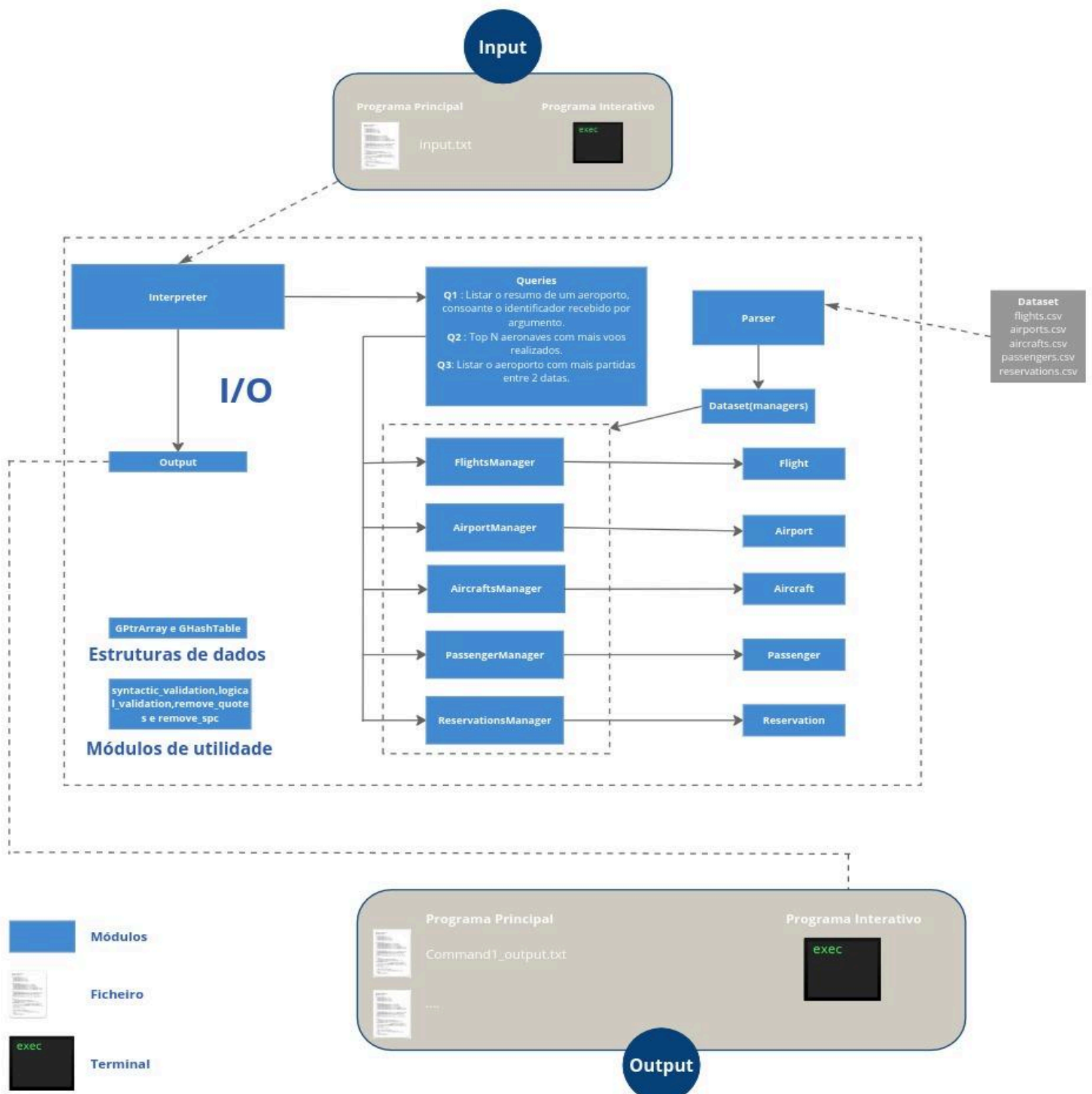
Após a extração, estes dados são submetidos a um processo de validação. A validação sintática, implementada no módulo 'syntactic_validation', verifica a conformidade dos dados com os formatos esperados, como datas no padrão "aaaa-mm-dd", endereços de email válidos, códigos de aeroporto IATA com três letras maiúsculas, entre outros. Paralelamente, a validação lógica, no módulo 'logical_validation', garante a consistência semântica dos dados, assegurando, por exemplo, que o aeroporto de destino de um voo é diferente do de origem, que uma aeronave referenciada num voo existe de facto no sistema, ou que os tempos de chegada são coerentes com os de partida.

Os registos que passam com sucesso nesta dupla validação são então encapsulados em objetos das respectivas entidades (voos, aeroportos, etc.) e armazenados em estruturas de dados geridas por módulos específicos ('flights_manager',

'airports_manager', etc.). Estes gestores são responsáveis pelo armazenamento e pela disponibilização dos dados para operações futuras.

A inovação central da nossa arquitetura é a forma como os dados válidos são armazenados. Em vez de listas ou arrays, utilizamos "hash tables" da biblioteca GLib como estrutura de armazenamento principal nos gestores de entidades. Por exemplo, o 'FlightsManager' utiliza uma hash table onde a chave é o 'flight_id' e o valor é a estrutura 'Flight' correspondente. A mesma abordagem foi aplicada a aeroportos (indexados pelo código IATA), aeronaves (pelo tail number), passageiros (pelo número de documento) e reservas (pelo reservation id). Esta decisão garante que operações de pesquisa, como verificar a existência de uma aeronave para um voo ou de um passageiro para uma reserva, sejam realizadas de forma muito mais eficiente.

A figura abaixo ilustra de forma simplificada a arquitetura implementada:



4. Discussão e Análise Crítica

A decisão de implementar “hash tables” desde a primeira fase revelou-se acertada. Ao contrário de uma solução baseada em arrays, que exigiria pesquisas sequenciais ($O(n)$) para validações de existência, a nossa abordagem reduz estas operações a $O(1)$ em casos médios. Isto foi crucial para a validação lógica, onde é frequente a necessidade de verificar se uma chave (ex: ‘aircraft_id’, ‘document_number’) existe noutra entidade. O impacto no desempenho do carregamento inicial é notório, especialmente à medida que o volume de dados aumenta.

A integração da GLib trouxe benefícios: para além das “hashtables”, a GLib fornece funções para manipulação de strings e memória que foram aproveitadas noutras partes do sistema. A gestão de memória, sempre um aspeto crítico em C, foi facilitada pelo uso de funções

como ‘g_hash_table_new_full’, que permite associar funções de destruição às chaves e valores, ajudando a prevenir memory leaks. A ferramenta ‘valgrind’ foi usada extensivamente para validar a correta libertação de todos os recursos.

A nível de implementação, o parser foi um dos módulos mais complexos e demorados.

A função ‘remove_quotes’ e ‘remove_spc’ mostrou-se vital para a limpeza dos dados lidos devido à necessidade de lidar com campos entre aspas, listas no formato CSV e remover caracteres extras.

O módulo ‘programa-testes’ foi desenvolvido para comparar automaticamente os ficheiros de output gerados com os resultados esperados. A função ‘compare_files’ realiza uma comparação linha a linha, reportando a primeira diferença encontrada. Adicionalmente, o módulo inclui medições de tempo de execução e, onde suportado, de uso de memória, utilizando ‘clock_gettime’ e ‘getrusage’, respetivamente.

5. Conclusão

Em conclusão, a Fase 1 do projeto foi concluída com sucesso, tendo-se não apenas cumprido os requisitos pedidos pelos docentes. A opção pelas “hash tables” da GLib desde o início revelou-se uma decisão fundamental, proporcionando ganhos de desempenho significativos e uma gestão de memória mais robusta durante a fase de carregamento e validação dos dados.

As principais lições aprendidas incluem a importância de selecionar as estruturas de dados certas antes da implementação e a eficácia de uma arquitetura modular para gerir a complexidade de um sistema com múltiplas entidades e relações.

Como trabalho futuro, planeia-se aprofundar a análise de desempenho, melhorando os tempos de execução e o aprimoramento do projeto em geral.